

Proposal: Replace the sourcing reverse block scan with an index lookup

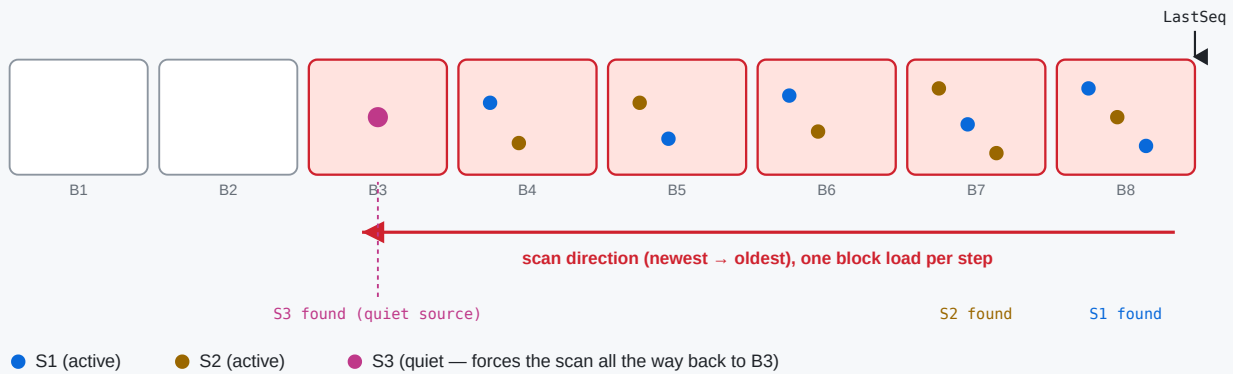
Status	Proposal
Area	JetStream stream sourcing (<code>server/stream.go</code>) + store index (<code>server/filestore.go</code> , <code>server/memstore.go</code>)
Date	2026-06-06
Companion	<code>sourcing-durable-resume.md</code> (lifecycle, durable consumer, triggers)
Target	<code>startingSequenceForSources</code> (<code>stream.go:4694</code>) and <code>setStartingSequenceForSources</code> (<code>stream.go:4566</code>)

1. The problem in one paragraph

On hub-stream **leader election / restart**, `setupSourceConsumers` calls `startingSequenceForSources` **unconditionally** (`stream.go:4821`). That function reconstructs each source's last sourced sequence by walking the stream's **own store backwards from `LastSeq`**, loading (decrypting, decompressing) one message block at a time, reading the `Nats-Stream-Source` header off each message, until **every** source has been located. A single quiet/sparse source forces the walk back across every newer block — worst case the **entire store**, under `mset.mu`.

The reverse block scan: rebuild each source's last sequence from stored headers

startingSequenceForSources walks back from LastSeq, LoadPrevMsgMulti loads (+decompresses) a block at a time until every source is found.



Why it can be slow on a limits-based stream

A single quiet/sparse source (S3) forces the scan past every newer block — worst case the entire stream. Each uncached block is read from disk, decrypted and decompressed. With many sources and a large store, leader election stalls under mset.mu while this runs.

2. The key insight

The store **already maintains a per-subject index** that can return the last sequence for a subject *without* walking messages:

- `fs.psim` — per-subject info, including `lblk` = the **last block index** that contains each subject (`filestore.go:3843-3852` , `8980-8984`).
- `mb.fss` — per-block subject state with `Last` per subject (`filestore.go:9003-9015`).

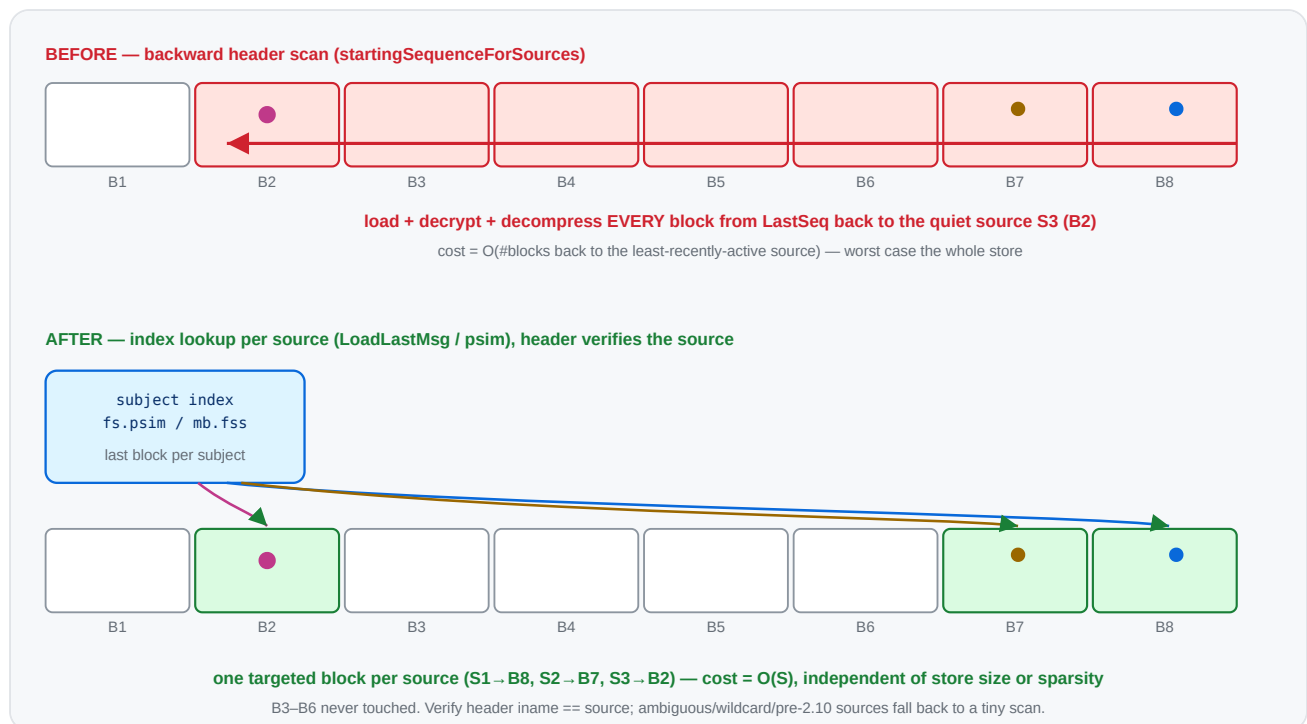
This is what powers `LoadLastMsg(subject) → loadLast ( filestore.go:8939 )`: for a literal subject it jumps straight to `info.lblk` and reads that subject's last message — **one targeted block load**, regardless of how far back it is. `MultiLastSeqs ( filestore.go:3806 )` does the same for a batch of filters in a single index pass.

The current scan ignores this index and instead does a generic backward message walk. The header it needs (the **origin** sequence) is right there in the message that `LoadLastMsg` already returns.

Why a plain "last seq for subject" isn't enough on its own: `si.sseq` must be the **origin** stream sequence taken from the `Nats-Stream-Source` header, not the hub's storage sequence. So we use the index to *find* the source's last stored message in $O(1)$, then read the origin seq from its header.

3. Proposed algorithm

Two phases. Phase 1 resolves the common case with index lookups; Phase 2 keeps today's scan only for sources that genuinely can't be attributed by subject.



For each source, its **stored subject(s)** are: the `FilterSubject` for a plain source, or the transform **Destination(s)** for a transformed source (this mirrors how the current sublist is built at `stream.go:4742-4757`).

Phase 1 – index lookup (per source, $O(1)$ targeted load):

```
for each source si with concrete stored subject sf (not "" / not ">"):  
    sm := store.LoadLastMsg(sf)           // jumps via psim, ~1 block  
    if sm has a JSStreamSource header:  
        (_, iname, sseq) := streamAndSeq(header)  
        if iname == si.iname:              // self-verifying attribution  
            si.sseq = sseq; mark resolved  
// anything not resolved (no header / header for a different source / wildcard)
```

Phase 2 – fallback reverse scan (only for the unresolved subset):

```
run today's LoadPrevMsgMulti loop, but with the sublist built from ONLY the  
unresolved sources, so it narrows immediately and stops as soon as they are found
```

The header check makes Phase 1 **self-correcting**: if a subject is shared between sources, or overlaps a direct publish on the hub, or the last message has no source header, the lookup simply doesn't match and that source drops to Phase 2 — never producing a wrong answer.

4. Before → after (code)

Before (`startingSequenceForSources` , condensed — `stream.go:4694`)

```
mset.resetSourceInfo()
// build a sublist of ALL sources' subjects
refreshSublist() // every source
for last := state.LastSeq; ; { // walk the WHOLE store backwards
    sm, seq, err := mset.store.LoadPrevMsgMulti(sl, last, &smv) // loads/decompresses
    if err == ErrStoreEOF || err != nil { break }
    last = seq - 1
    ...
    _, iName, sseq := streamAndSeq(bytesToString(ss))
    update(iName, sseq) // narrows sublist as sources are processed
    if len(seqs) == expected { return } // stop only when ALL sources are processed
}
```

Cost: O(blocks back to the least-recently-active source) block loads → worst case the entire store.

After

```
mset.resetSourceInfo()
var smv StoreMsg
unresolved := map[string]*sourceInfo{}

// Phase 1: targeted index lookup per source.
for _, ssi := range mset.cfg.Sources {
    si := mset.sources[ssi.iname]
    if si == nil { continue }
    subj := storedSubjectForSource(ssi) // FilterSubject, or transform
    if subj == _EMPTY_ || subj == fwcs || subjectHasWildcard(subj) {
        unresolved[ssi.iname] = si // can't attribute by a single subject
        continue
    }
    sm, err := mset.store.LoadLastMsg(subj, &smv) // O(1) via psim/fss, returns nil if not found
    if err != nil || sm == nil || len(sm.hdr) == 0 {
        unresolved[ssi.iname] = si
        continue
    }
}
```

```

    }
    if ss := sliceHeader(JSStreamSource, sm.hdr); len(ss) > 0 {
        if _, iName, sseq := streamAndSeq(bytesToString(ss)); iName == ssi.iname
            si.sseq, si.dseq = sseq, 0 // verified – resolved without
            continue
        }
    }
    unresolved[ssi.iname] = si
}

// Phase 2: only the leftovers pay the (now tiny) reverse scan.
if len(unresolved) > 0 {
    mset.reverseScanForSources(unresolved) // = today's loop, scoped to
}

```

Cost: Phase 1 = $O(S)$ targeted lookups (recent sources share/cache the last block; a quiet source is **one** targeted load, not a walk). Phase 2 only runs for ambiguous sources, with a smaller sublist.

Multi-destination transform sources: call `LoadLastMsg` for each destination and keep the highest origin `sseq`, or defer them to Phase 2. Either is fine; deferring is simplest to start.

5. Before → after (complexity)

	Before	After
Index used	none (generic message walk)	<code>psim</code> / <code>fss</code> per-subject last-block index
Blocks loaded	back to least-active source ($\leq$ all blocks)	~1 per source, only the blocks actually holding a source's last msg
Sensitivity to a quiet source	drags the scan back across the whole store	none — jumps straight to its block
Sensitivity to store size	linear in blocks scanned	none
Worst case still possible?	normal case	only if every source is <code>> /wildcard/shared-subject/pre-2.10</code> (→ Phase 2 == today)

	Before	After
Held lock	<code>mset.mu</code> for the whole walk	<code>mset.mu</code> for O(S) lookups; Phase 2 only for leftovers

The edge → hub topology (each edge mapped to a distinct subject/domain) lands entirely in Phase 1 → the backward scan disappears for that case.

6. Correctness & edge cases

- **Same semantics.** Today's scan records, per source, the **most recent** stored message's origin seq (first hit scanning backward). `LoadLastMsg` returns exactly that message. Deleted/interior messages are handled by `loadLast` (it skips `dmap` entries and walks to the previous block if needed), matching the scan's behaviour.
- **Shared subjects / direct publishes.** Resolved safely by the header-iname verification → fall to Phase 2.
- **Wildcard / empty (>) filters.** Can't be attributed to one source by subject → Phase 2 (same as today for them).
- **Pre-2.10 headers** (no `iname`, only stream name): verification won't match → Phase 2, which keeps the existing stream-name matching path.
- **memstore.** `MemStore` has the simpler linear `LoadPrevMsgMulti`; `LoadLastMsg` there is also index-light, but memstore streams are small, so Phase 2 cost is negligible. (We can add a memstore `loadLast` fast path later if needed.)
- **setStartingSequenceForSources** (the `STREAM.UPDATE` path, `stream.go:4566`) gets the same Phase 1 treatment for the subset of sources it processes.

7. Risks

- **Index freshness.** `psim / fss` may lazily need a recalculation (`lastNeedsUpdate`, `recalculateForSubj`); `loadLast / MultiLastSeqs` already handle that, so we inherit correct behaviour.
- **Behavioural parity.** Phase 1 must produce identical `si.sseq` to the old scan for non-ambiguous sources; this is the core test (below). Keeping Phase 2 byte-for-byte as today bounds the blast radius.
- **Encryption/compression.** Phase 1 still loads the one block holding a source's last message (decrypt/decompress), but only that block — no change in correctness, large reduction in volume.

8. Testing & validation

- **Equivalence test:** build a stream sourcing from K origins with distinct subjects, varying activity (some quiet); assert the new resolver yields the same `si.sseq` per source as the old scan.
- **Ambiguity test:** sources sharing a subject, a `>` source, a direct-publish overlap, and a pre-2.10 header → assert correct fallback and correct sequences.
- **Benchmark:** `BenchmarkStartingSequenceForSources` on a large filestore (e.g. 5 GB, many blocks) with one deliberately quiet source — compare blocks loaded / wall time before vs after. Expect the "quiet source" case to drop from "scan to the bottom of the store" to a single targeted load.
- Run with `-race`.

9. Relationship to the durable-consumer proposal

This change is **orthogonal and complementary** to the durable-consumer/ `si.sseq` -persistence ideas in `sourcing-durable-resume.md`:

- Durable consumers / persisted `si.sseq` aim to **skip** resume work entirely (when state survives).
- This proposal makes the **fallback resume itself cheap** — which still runs on cold start, on snapshot restore, for limits/clustered streams, and any time persisted state is missing.

Recommended order: land this index-based resume first (self-contained, no protocol/state change, helps every retention type today), then layer the persistence/durable improvements on top.

Appendix — key references

Symbol	File:line	Role
<code>startingSequenceForSources</code>	<code>stream.go:4694</code>	the scan to replace (Phase 1 + Phase 2)
unconditional call	<code>stream.go:4821</code>	runs on every leader election / restart
<code>setStartingSequenceForSources</code>	<code>stream.go:4566</code>	update-path twin, same treatment

Symbol	File:line	Role
sublist build (stored subjects)	<code>stream.go:4742-4757</code>	defines a source's stored subject(s)
<code>LoadLastMsg</code> / <code>loadLast</code>	<code>filestore.go:9048</code> / <code>8939</code>	index-based last-msg-for-subject (returns header)
<code>MultiLastSeqs</code>	<code>filestore.go:3806</code>	batched last-seq-per-filter via index
<code>fs.psim</code> (last block per subject)	<code>filestore.go:8980</code>	the index Phase 1 exploits
<code>LoadPrevMsgMulti</code>	<code>filestore.go:9403</code> / <code>memstore.go:1991</code>	the backward walk used by Phase 2
<code>streamAndSeq</code>	parses <code>Nats-Stream-Source</code>	origin stream/iname/seq from header